

PROJECT REPORT

USER AUTHENTICATION SYSTEM

Submitted By:

Prateek Kumar

Bachelor of Technology (B.Tech)
Artificial Intelligence & Machine Learning
Academic Year: 2025-2026

INDEX

Sr. No	Contents	Page No
1.	Introduction	3
2.	Project Objective	4
3.	Technologies Used	5
4.	Project Features	6
5.	System Requirements	7
6.	Project Structure	8
7.	Database Setup	9
8.	How to Run the Project	10-11
9.	Working of the Project	12-13
10.	Authentication Flow	14
11.	Screenshots	15-16
12.	Challenges Faced	17
13.	Learning Outcomes	18
14.	Future Improvements	19
15.	Conclusion	20
16.	Acknowledgements	21

Introduction

The User Authentication System is a full-stack web development project created to provide secure user registration and login functionality. The main purpose of this project is to understand how authentication systems work in modern web applications.

This project allows users to create an account, log in securely, access a protected dashboard page, and log out safely. The project is developed using HTML, CSS, JavaScript, Node.js, Express.js, and MySQL.

During the development of this project, important backend concepts such as routing, database connectivity, session management, password hashing, and protected routes were learned and implemented.

The frontend of the project is designed using simple and clean user interfaces, while the backend handles authentication logic and database operations securely.

This project helped in understanding the complete flow of a full-stack authentication system from frontend forms to backend processing and database storage.

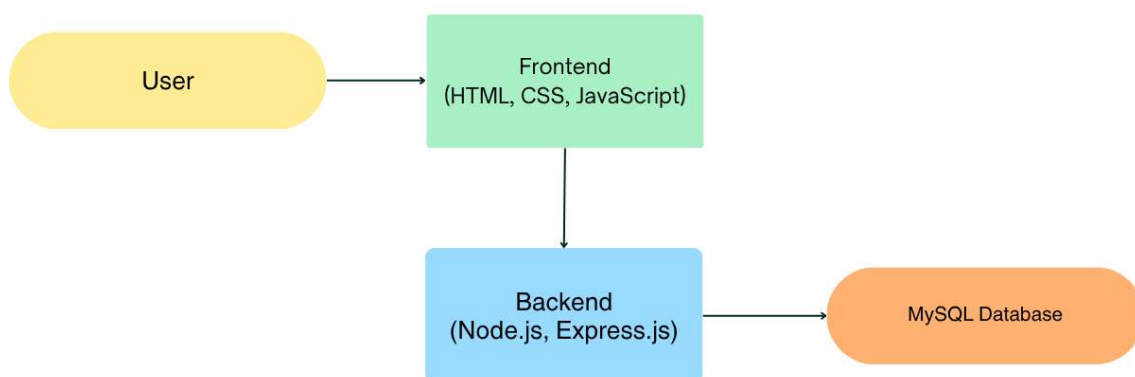


Figure 1: Basic Architecture of User Authentication System

Project Objective

The main objective of this project is to develop a secure user authentication system using full-stack web development technologies. The project aims to understand how frontend, backend, and database systems work together in a real-world web application.

The objectives of this project are:

- To create a user registration system
- To implement secure user login functionality
- To connect Node.js backend with MySQL database
- To understand session-based authentication
- To create protected routes that only logged-in users can access
- To implement logout functionality
- To learn password hashing using bcryptjs
- To gain practical knowledge of full-stack web development

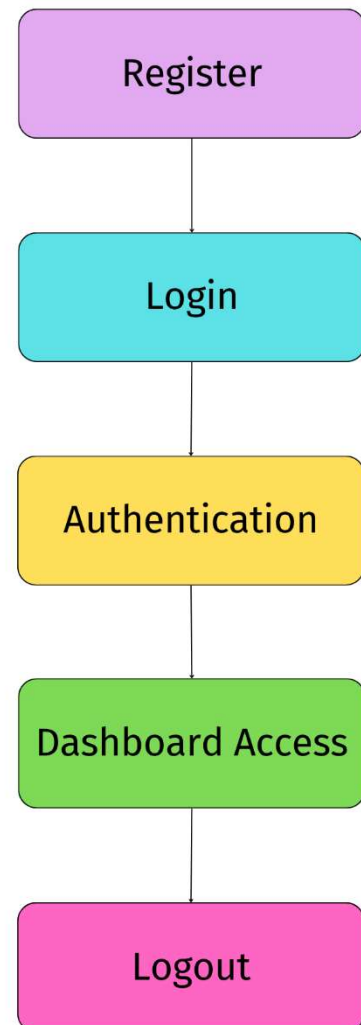


Figure 2: Basic Authentication Flow

Technologies Used

Different frontend, backend, and database technologies were used during the development of this project. These technologies helped in building the complete authentication system with secure login and registration functionality.

Technology	Purpose
HTML	Used to create the structure of web pages
CSS	Used for styling and designing the user interface
JavaScript	Used for frontend functionality and interactions
Node.js	Used to run JavaScript on the server side
Express.js	Used to create backend routes and server logic
MySQL	Used to store user data in database
bcryptjs	Used for password hashing and security
express-session	Used for session management and authentication
VS Code	Used as code editor for development
Git & GitHub	Used for version control and project management

Project Features

The User Authentication System contains different features related to user registration, login security, session management, and protected access. These features help in creating a secure and functional authentication system.

- **User Registration System**

New users can create an account by entering username, email, and password information.

- **User Login Authentication**

Registered users can log in using valid email and password credentials.

- **Password Hashing**

Passwords are encrypted using bcryptjs before storing them in the database for better security.

- **Session Management**

User sessions are created after successful login to maintain authentication status.

- **Protected Dashboard Route**

Only authenticated users can access the dashboard page.

- **Logout Functionality**

Users can securely log out and their session gets destroyed.

- **MySQL Database Integration**

All user information is stored and managed using MySQL database.

- **Responsive User Interface**

The frontend pages are designed using simple and clean HTML and CSS.

System Requirements

The following hardware and software requirements were needed to develop and run the User Authentication System project successfully.

Software Requirements

Software	Purpose
Visual Studio Code	Code editor used for development
Node.js	Used to run backend server
MySQL Workbench	Used for database management
Google Chrome	Used to test the web application
Git & GitHub	Used for version control

Hardware Requirements

Hardware	Requirement
Processor	Intel i3 or above
RAM	Minimum 4 GB
Storage	Minimum 500 MB free space
Internet Connection	Required for package installation

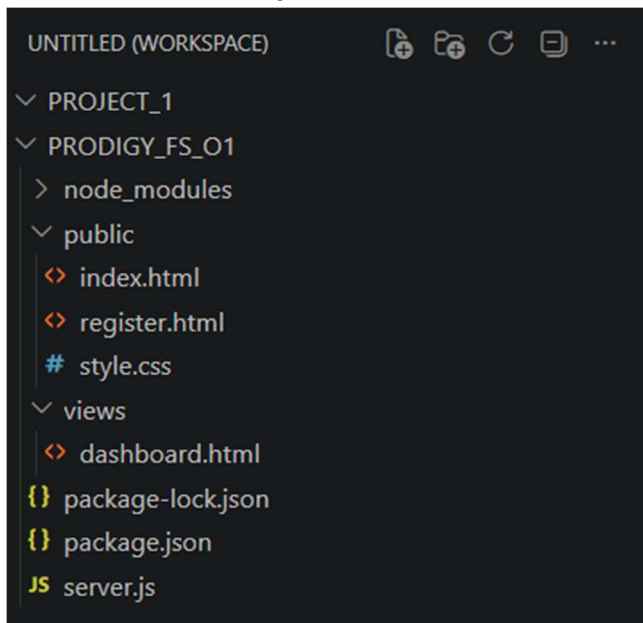
- Operating System Used: Windows 10/11
- Backend Port Number: 3000
- Database Server: MySQL

Project Structure

The project is divided into different folders and files for frontend pages, backend server, and protected dashboard functionality. This structure helps in managing the project files in an organized manner.

MAIN PROJECT STRUCTURE

Authentication Project



File / Folder	Description
public	Contains frontend files
index.html	Login page
register.html	User registration page
style.css	Styling for all frontend pages
views	Contains protected pages
dashboard.html	Dashboard page after login
server.js	Main backend server file
package.json	Stores project dependencies
node_modules	Contains installed npm packages

The frontend files are stored inside the public folder, while the protected dashboard page is stored inside the views folder. The server.js file handles all backend routes, authentication logic, database connectivity, and session management.

Database Setup

MySQL database was used in this project to store user registration and login information. The database stores user details such as username, email, and encrypted password securely.

Creating Database

```
CREATE DATABASE authentication_system;
```

This query creates a new database named authentication_system which is used in the project.

Selecting Database

```
USE authentication_system;
```

This command selects the authentication_system database for further operations.

Creating Users Table

```
CREATE TABLE users (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  username VARCHAR(100),  
  email VARCHAR(100),  
  password VARCHAR(255)  
);
```

The users table stores all registered user information including username, email, and encrypted password.

TABLE DESCRIPTION

Field Name	Description
id	Unique ID for each user
username	Stores username
email	Stores user email
password	Stores encrypted password

The backend server was connected with MySQL database using the mysql2 package. The connection was established inside the server.js file using localhost, root username, and MySQL password.

How to Run the Project

The following steps were followed to run the User Authentication System project successfully on the local computer system.

Step 1: Open Project Folder

Open the Authentication_Project folder using Visual Studio Code.

Step 2: Open Terminal

Open the integrated terminal in Visual Studio Code using:

`Ctrl + ``

Step 3: Install Required Packages

`npm install`

This command installs all required npm packages and dependencies used in the project.

Step 4: Start MySQL Server

Open MySQL Workbench and make sure the MySQL server is running properly before starting the backend server.

Step 5: Start Backend Server

`node server.js`

Expected Output:

Connected to MySQL database
Server started on port 3000

This message confirms that the backend server and MySQL database connection are working successfully.

Step 6: Open Browser

Open the following URL in browser:

<http://localhost:3000>

The login page of the User Authentication System will open in the browser.

Step 7: Register User

Click on "Register Here" and create a new user account by entering username, email, and password.

Step 8: Login into System

Login using the registered email and password credentials.

STEP 9 – ACCESS DASHBOARD

After successful login, the user is redirected to the protected dashboard page.

Step 10: Logout

Click on the Logout button to destroy the session and securely log out from the system.

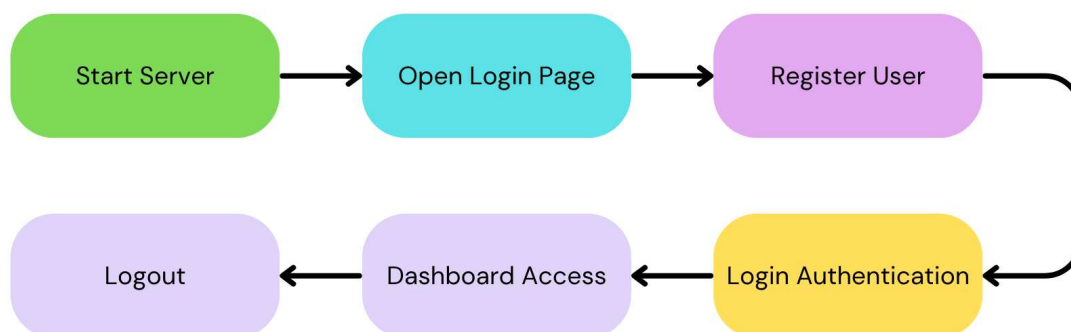


Figure 3: Project Execution Flow

Working of the Project

The User Authentication System works by connecting frontend pages, backend server logic, and MySQL database together. The project follows a complete authentication process including user registration, login verification, session management, dashboard access, and logout functionality.

User Registration

The registration page allows new users to create an account by entering username, email, and password information. When the user submits the form, the frontend sends the data to the Express.js backend server.

The backend receives the form data and hashes the password using bcryptjs before storing it inside the MySQL database. After successful registration, the user information is stored securely in the users table.

User Login

The login page allows registered users to log in using their email and password. The backend checks whether the entered email exists in the database.

The entered password is compared with the hashed password stored in the database using bcryptjs. If both values match correctly, the login process becomes successful.

Session Management

After successful login, a session is created using express-session. The session helps the server remember the authenticated user while accessing protected pages.

Session management improves authentication security and prevents unauthorized access to protected routes.

Protected Dashboard Access

The dashboard page is protected using session authentication. Only logged-in users with valid sessions can access the dashboard route.

If an unauthorized user tries to access the dashboard directly without login, the backend blocks access and displays an authentication message.

Logout Functionality

The logout feature ends the current user session and redirects the user to the login page, preventing access to the dashboard until login again.

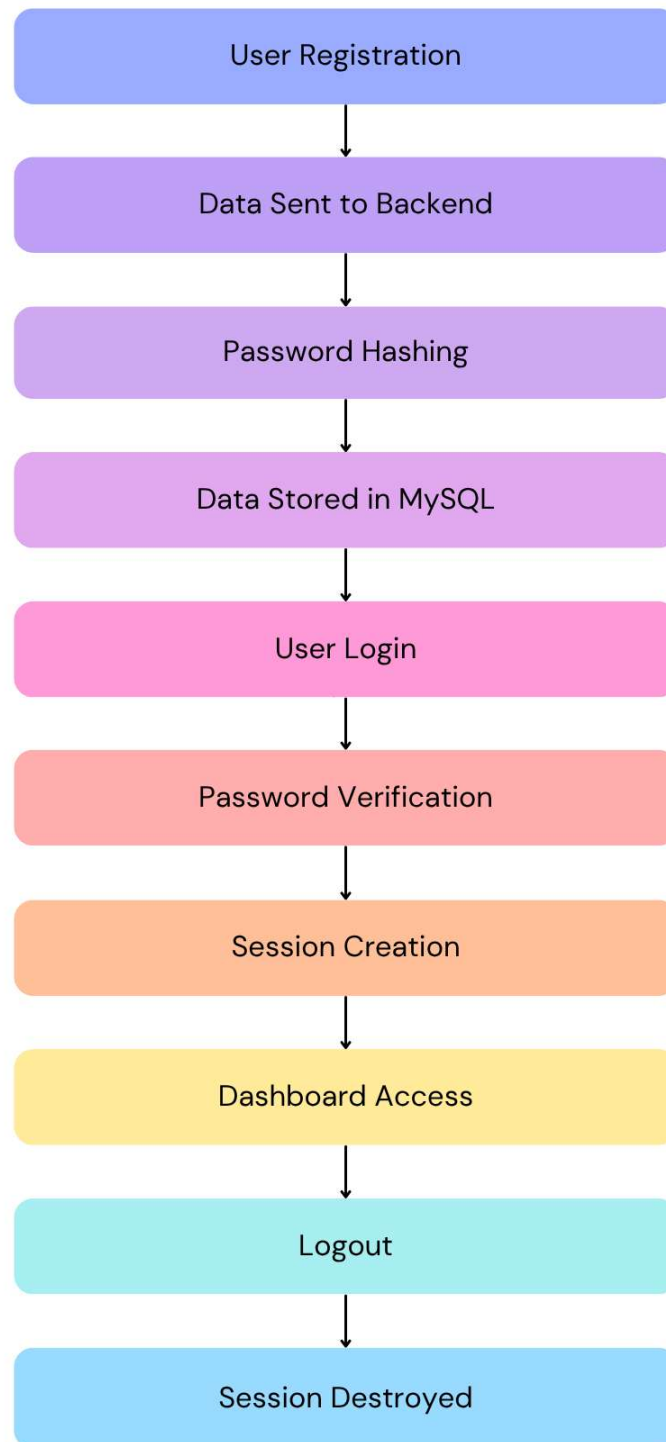


Figure 4:Complete Authentication Flow

The project demonstrates the complete workflow of a basic full-stack authentication system using frontend technologies, backend development, session management, and database integration.

Authentication Flow

The authentication flow explains how user data moves through the system during registration, login, dashboard access, and logout processes. The project uses frontend forms, backend routes, sessions, and MySQL database together to perform authentication securely.

1. User Registration Flow

When a new user enters username, email, and password on the registration page, the frontend sends the data to the backend server using a POST request.

The backend receives the user data and hashes the password using bcryptjs before storing it securely inside the MySQL database.

2. User Login Flow

The user enters login credentials on the login page. The backend checks whether the email exists in the database.

The entered password is compared with the hashed password stored inside the database using bcryptjs password comparison.

3. Session Authentication

After successful login, a session is created using express-session. The session stores the authenticated user information temporarily and helps the backend identify logged-in users

4. Protected Dashboard Access

The dashboard route is protected using session verification. Before opening the dashboard page, the backend checks whether the user session exists.

If the session is valid, dashboard access is granted. Otherwise, unauthorized access is blocked.

5. Logout Process

When the user clicks the logout button, the session gets destroyed and the user is redirected back to the login page. After logout, the dashboard cannot be accessed without logging in again.

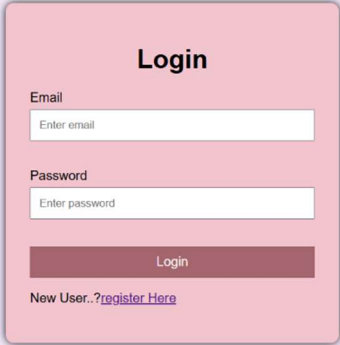
The authentication flow demonstrates how frontend pages, backend server logic, database operations, session management, and password security work together to create a secure authentication system.

Screenshots

The following screenshots show the working interface and functionality of the User Authentication System project developed using HTML, CSS, JavaScript, Node.js, Express.js, and MySQL.

1. Login Page

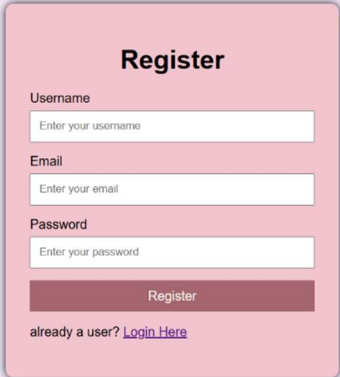
The login page allows registered users to enter their email and password credentials for authentication.



The screenshot shows a login form titled "Login" centered on a light purple background. The form is a light pink rectangle with rounded corners. It contains two input fields: "Email" with a placeholder "Enter email" and "Password" with a placeholder "Enter password". Below these fields is a dark red "Login" button. At the bottom of the form, there is a link that says "New User.?register Here".

2. Register Page

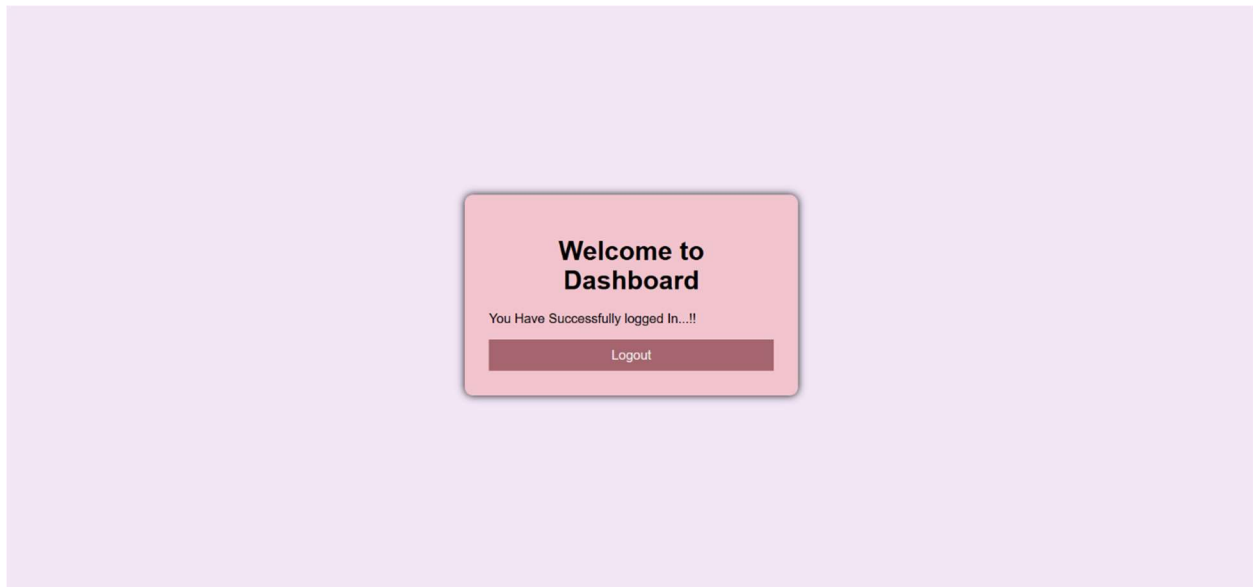
The registration page allows new users to create an account by entering username, email, and password details.



The screenshot shows a registration form titled "Register" centered on a light purple background. The form is a light pink rectangle with rounded corners. It contains three input fields: "Username" with a placeholder "Enter your username", "Email" with a placeholder "Enter your email", and "Password" with a placeholder "Enter your password". Below these fields is a dark red "Register" button. At the bottom of the form, there is a link that says "already a user? Login Here".

3. Dashboard Page

The dashboard page is a protected page that becomes accessible only after successful user login.



4. MySQL Database Table

The users table stores registered user information including username, email, and encrypted password.

id	username	email	password
1			\$2b\$10\$VrxOFFub0OzjodMWHTR8.aZdB7V2GkclMZ.Ug9JGy2xJ6sUz6mN2
2	Prateek	Prateek@gmail.com	\$2b\$10\$PH3ztliKiCCEmTqihFhudMOvpvZQfjqJXfpA78KvKsawkzk/P8CbK
3	a	a@gmail.com	\$2b\$10\$2vzt2CC3UInmvq/EfZhiV5.Gr13a454fK.RJSJOdyC0tPM.4J2MJvy
4	b	b@gmail.com	\$2b\$10\$eS258BURtfGycWn9lia8.GDtpwqkyR6ZJM2Ow2w0jQjthWJ3cb8X.
5	c	c@gmail.com	\$2b\$10\$K1akBKfFiYCXGngGdo1Z4O6osJOL070pngw6Nel3domLJN0xLJkik
6	d	d@gmail.com	\$2b\$10\$IYM.Utu.DY4PFHP5yurn3.Guqc1pkc7osB54Xy7SKTF8yNZlpEBmy
7	e	e@gmail.com	\$2b\$10\$WFm/wejwgJD54nkZa8GD4OEQcF8ejWij4NwCAF0Xo7ZyOPYBVEa
8			

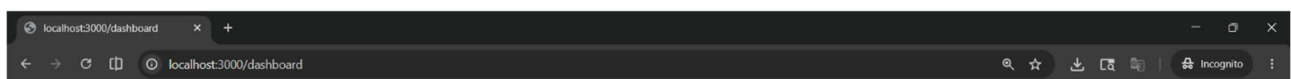
5. Backend Server Output

The terminal output confirms successful database connection and backend server execution.

```
node - PRODIGY_FS_01
PS C:\Users\LENOVO\Desktop\PRODIGY_FS_01> node server.js
Server started on port 3000
Connected to MySQL database
```

6. Protected Route Block

Unauthorized Dashboard Access Blocked



Please login first

Challenges Faced

During the development of the User Authentication System project, different challenges were faced while connecting the frontend, backend, and database components together. These challenges helped in improving problem-solving and debugging skills during the project development process.

1. Database Connection Issues

Initially, difficulty was faced while connecting the Node.js backend with the MySQL database because of incorrect connection details and server configuration problems.

2. Understanding Express Routes

It was challenging to understand the difference between GET routes and POST routes during the initial stages of backend development.

3. Session Management Confusion

Implementing session-based authentication using express-session was difficult in the beginning because understanding how sessions work required backend knowledge.

4. Password Hashing Implementation

Using bcryptjs for password hashing and password comparison during login created confusion initially because encrypted passwords cannot be directly compared with normal text passwords.

5. Protected Route Access

Protecting the dashboard route and preventing unauthorized users from accessing it without login required additional backend logic and session verification.

6. Frontend and Backend Integration

Connecting HTML forms with backend routes and handling form data properly was one of the major learning challenges during development.

7. Debugging Errors

Different server errors, routing issues, and login problems occurred during development and were solved through testing and debugging.

Although several challenges were faced during development, solving these problems helped in understanding the practical implementation of full-stack authentication systems more effectively.

Learning outcome

The development of the User Authentication System project helped in gaining practical knowledge of frontend development, backend development, database integration, and authentication systems. Different technical concepts were learned during the implementation of this full-stack project.

1. Learned Frontend Development

Improved understanding of HTML, CSS, and JavaScript for designing user interfaces and handling form inputs.

2. Learned Backend Development

Understood how Node.js and Express.js are used to create backend servers and handle routing functionality.

3. Learned Database Connectivity

Learned how to connect MySQL database with Node.js using the mysql2 package.

4. Learned Authentication System

Understood the complete working process of user registration, login authentication, and protected routes.

5. Learned Session Management

Gained knowledge about session-based authentication using express-session for maintaining user login state.

6. Learned Password Security

Learned how bcryptjs is used for password hashing and secure password verification.

7. Improved Debugging Skills

Solved multiple frontend and backend errors during development which improved debugging and problem-solving abilities.

8. Understood Full-Stack Workflow

Learned how frontend, backend, and database components interact together in a real-world web application.

This project provided practical exposure to full-stack web development concepts and helped in understanding the implementation of secure authentication systems using modern web technologies.

Future Improvements

Although the User Authentication System project was developed successfully with core authentication features, several improvements and additional functionalities can be implemented in the future to enhance the project further.

1. Forgot Password Functionality

A forgot password feature can be added to allow users to reset their passwords securely.

2. Email Verification System

Email verification can be implemented during user registration for better account security.

3. Improved User Interface

The frontend design can be improved further using advanced CSS frameworks and responsive layouts.

4. Profile Management System

Users can be given the ability to edit their profile information after login.

5. Admin Panel

An admin dashboard can be developed to manage users and monitor system activities.

6. Better Security Features

Additional security measures such as OTP verification and login attempt limitations can be implemented.

7. Deployment on Cloud Platforms

The project can be deployed on hosting platforms such as Render, Railway, or Heroku for public access.

8. Mobile Responsive Design

The application can be optimized further for mobile devices and different screen sizes.

These future improvements can help in making the authentication system more advanced, secure, user-friendly, and suitable for real-world applications.

Conclusion

The User Authentication System project was developed successfully using HTML, CSS, JavaScript, Node.js, Express.js, and MySQL. The project demonstrated the complete implementation of user registration, login authentication, session management, protected dashboard access, and logout functionality.

The project helped in understanding how frontend, backend, and database technologies work together in a full-stack web application. Important concepts such as routing, database connectivity, password hashing using bcryptjs, and session-based authentication were implemented successfully during the development process.

The project also improved practical knowledge of backend development, debugging, authentication systems, and database operations. Different challenges faced during the development process helped in gaining better problem-solving and troubleshooting skills.

Overall, this project provided valuable learning experience in full-stack web development and helped in understanding the practical implementation of secure authentication systems used in modern web applications.

Acknowledgements

I would like to express my sincere gratitude to Prodigy InfoTech for providing me with the opportunity to complete this virtual internship and improve my practical web development skills.

I would also like to thank the following platforms, tools, and resources that helped me throughout the project development process:

- HTML Official Documentation = <https://html.spec.whatwg.org/>
- CSS Official Documentation = <https://www.w3.org/Style/CSS/>
- JavaScript Documentation (MDN) = <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- Node.js Official Website = <https://nodejs.org/>
- Express.js Official Website = <https://expressjs.com/>
- MySQL Official Website = <https://www.mysql.com/>
- bcryptjs Package Docs = <https://www.npmjs.com/package/bcryptjs>
- Git Official Website = <https://git-scm.com/>
- GitHub Platform = <https://github.com/>
- Claude AI Assistant = <https://claude.ai/>
- GitHub Copilot = <https://github.com/features/copilot>
- YouTube Learning Platform = <https://www.youtube.com/>
- Gmail Communication Service = <https://mail.google.com/>
- npm Package Registry = <https://www.npmjs.com/>

I sincerely appreciate all these resources for their valuable support, documentation, tutorials, and guidance that contributed to the successful completion of this project.

THANKYOU